

## Week 10 — Functions: Math Machines

Name: \_\_\_\_ Date: \_\_\_\_

@ **builds** a machine · calling **runs** it · the **output slot** hands the answer back. Math's  $f(3)$  and MATLAB's  $f(3)$  are the same thing.

### Today's words

| Word                 | What it means                                                                                |
|----------------------|----------------------------------------------------------------------------------------------|
| <b>function</b>      | a machine: input in the slot, output out the chute                                           |
| @                    | builds a machine: <code>f = @(x) 2*x + 1</code> (building it doesn't run it!)                |
| <b>function file</b> | a named machine's own <code>.m</code> file, named after itself                               |
| <b>parameter</b>     | the input slot's name — the <code>x</code> in <code>@(x)</code> or <code>double_it(x)</code> |
| <b>call</b>          | using the machine: <code>double_it(5)</code> — parentheses press the button                  |
| <b>output slot</b>   | the <code>out =</code> in a function line — it hands the answer back                         |

### 1 • Be the machine 🤖

Fill each table by running the machine in your head.

```
% triple.m          % machine.m
function out = triple(x)  function out = machine(x)
    out = x * 3;          out = x * x - 1;
end                    end
```

| in (x) | triple(x) |  | in (x) | machine(x) |
|--------|-----------|--|--------|------------|
| 1      |           |  | 2      |            |
| 2      |           |  | 3      |            |
| 5      |           |  | 4      |            |
| 10     |           |  | 10     |            |

### 2 • Match the call to its output 🖱️

Draw a line from each call to what it prints. One output is left over!

```
% double_it.m      % gap.m
function out = double_it(x)  function out = gap(a, b)
    out = x * 2;            out = a - b;
end                      end
```

| Call                                       |  | Output |
|--------------------------------------------|--|--------|
| <code>disp(double_it(6))</code>            |  | -5     |
| <code>disp(gap(9, 4))</code>               |  | 8      |
| <code>disp(gap(4, 9))</code>               |  | 12     |
| <code>disp(double_it(double_it(2)))</code> |  | 5      |
|                                            |  | 24     |

### 3 • Build a machine on paper 🛠️

Write a complete function **file** for a machine named `add_ten` that takes one number and hands back that number plus 10. (Three lines — don't forget the output slot on the function line, and the `end`. Bonus point: what must the file be named?)

### 🧠 Brain teaser (optional — take it home)

Two machines: `double_it = @(x) x * 2` and `add_three = @(x) x + 3`

Work **inside out**, like nested parentheses:

- `double_it(add_three(2)) = ____`
- `add_three(double_it(2)) = ____`
- Same machines, same input, both orders — same answer? Circle: **YES / NO**

In math language that's  $f(g(2))$  vs  $g(f(2))$ . Bring your answers next week for a shout-out — and find out what this has to do with putting on socks and shoes.